**Project Interim Report**

**On**

# File Compression Simulator
# (Using Core Java & OOP Concepts)

**RU-100-01-00012**

**Object Oriented Programming with Java**

**SESSION: 2025-26**

Submitted By


Student name : - SHANTANU KAUSHIK

Reg No. : - RU - 25 - 11333


Bachelor of Technology in Computer Science & Engineering in association with GOOGLE

Under the Guidance of

**Mr. Ashish Shivhare**

**JAVA Trainer & Expert**


**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

**RUNGTA INTERNATIONAL SKILLS UNIVERSITY**

**BHILAI, CG**


**(April 2026)**

## Abstract

Hotel reservation systems are widely used in the hospitality industry to manage room bookings, track availability, and calculate billing. This project focuses on building a Hotel Room Reservation Application using Object-Oriented Programming (OOP) principles such as classes, interfaces, encapsulation, and arrays. The system represents rooms using a Room class with an availability status and manages multiple rooms utilizing a 2D array. Furthermore, it uses a Service interface to define billing operations, successfully calculating the total stay cost based on room type, duration, and additional services.


## Introduction

**Objectives :**

- Represent rooms using a Room class with availability status.
- Manage multiple rooms using a 2D array (Room[][]) inside a Hotel class.
- Use a Service interface to define billing operations.
- Calculate total stay cost based on room type, duration, and additional services.

## Scope

Students will design a structured and scalable system that simulates real-world hotel booking logic. This project demonstrates the practical use of interfaces and abstraction, classes and objects, encapsulation, 2D arrays, method implementation, and system design.

## System Requirements

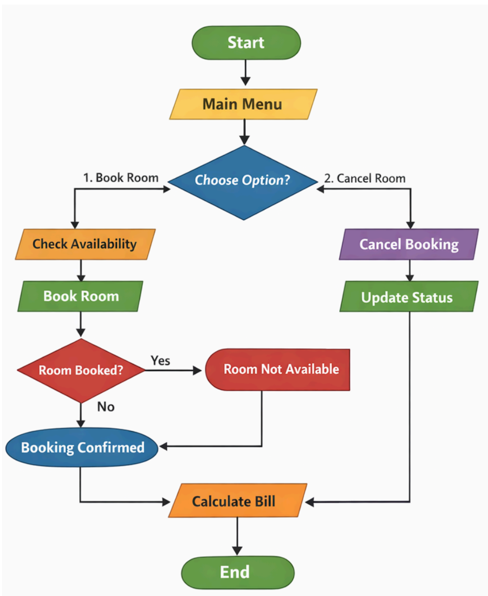
Hardware: Computer with 16 GB RAM

Software: Java JDK, IDE (VS Code/Eclipse/Edit Plus)

## Methodology

The project follows a step-by-step approach starting with the definition of a Service interface to handle billing. Next, a Room class is created to define room attributes, followed by a Hotel class that maintains a 2D array representing floors and rooms. Booking logic is then implemented to check availability and mark rooms as occupied, alongside cancellation logic to revert availability. Finally, the Service interface is implemented to calculate bills based on the days stayed and the room rate.

## Flowchart



Sample class diagram –

Service (Interface):

- Methods: calculateBill(int days, double rate)

Room (Class):

- Variables: roomNumber, roomType (Single/Double/Deluxe), pricePerDay, isAvailable (boolean) • Methods: Constructor to initialize values.

Hotel (Class):

- Variables: Room[][] rooms (2D array representing floors and rooms)
- Methods: Constructor (to initialize rooms), bookRoom(int floor, int roomIndex), cancelBooking(int floor, int roomIndex)

## Implementation

Service Interface:

- This interface declares the method required to calculate the final bill by multiplying the days by the rate.

Room Class:

- This class acts as a blueprint for individual rooms, encapsulating data like the room's number, type, price, and current availability status.

Hotel Class:

- This class serves as the core manager, utilizing a 2D array structure (Room[][]) to simulate the hotel's layout. It contains the logical methods to bookRoom (checking if isAvailable is true and then setting it to false) and cancelBooking (resetting isAvailable to true). It also implements the Service interface to process the final billing calculations, optionally factoring in extra charges like food or tax.

**Sample Input/Output**

- **Room booked successfully**
- **Room already occupied**
- **Total Bill for 3 days: 4500.0**
- **Room Status:**
- **Room 101 - Occupied**
- **Room 102 – Available**

## Gantt Chart

**1. Planning & Structure**

A Gantt chart helps you break the project into tasks (design, coding, testing, etc.) and set timelines. Even if it's already built once, rebuilding or modifying still needs planning.

**2. Time Management**

It shows:

- When each task starts
- When it should finish
- This helps avoid delays and last-minute panic.

## Conclusion

The project successfully demonstrates Java OOP concepts and provides a functional system. By completing this system, the fundamental principles of interface-based design and abstraction, 2D array handling, and state changes in objects are effectively applied to build a modular and scalable application simulating real-world booking logic.

## References

**[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.**
**[2] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com**